

Bài thuyết trình

TÍNH TOÁN BẤT ĐỒNG BỘ

Khám phá kiến trúc bất đồng bộ, concurrent.futures và asyncio.

01

KIẾN THỨC TỔNG QUAN

Tính toán bất đồng bộ, Non-blocking, Các mô hình thực thi và Quản lý CPU.

02

CONCURRENT.FUTURES

ThreadPool, ProcessPool, Futures, Callbacks và Quản lý Task Queue.

03

MODULE ASYNCIO

Event Loop, Coroutines, Tasks, và Kiến trúc Single-threaded Concurrency.

Phần 01

Kiến thức tổng quan

Nền tảng về tính toán bất đồng bộ và lập trình không chặn.

1. Định nghĩa tính toán bất đồng bộ

Tính toán bất đồng bộ (Asynchronous Computing) là một mô hình thực thi đồng thời mà trong đó các tác vụ được thực hiện **đan xen** và **không chặn** lẫn nhau trên cùng một trục thời gian.

Mô hình này có thể được vận hành dựa trên kiến trúc điều khiển bằng sự kiện thông qua:

- Một luồng kiểm soát duy nhất
- Một tập hợp nhiều luồng
- Các tiến trình độc lập

2. Lập trình không chặn (Non-blocking)

Khái niệm

- Là phương pháp ngăn các tác vụ I/O làm đóng băng luồng xử lý, cho phép hệ thống trả về phản hồi lập tức. Khi kết hợp với bất đồng bộ, chương trình có thể xử lý công việc khác trong lúc chờ kết quả, giúp tận dụng CPU hiệu quả hơn.

Đặc điểm chính

- Không dừng luồng chính khi chờ I/O.
- Nhiều tác vụ được xử lý xen kẽ.
- Thường chạy trên một luồng duy nhất nhưng vẫn xử lý nhiều việc.

Cơ chế

- Dựa trên mô hình bất đồng bộ
- Tự động tạm dừng & khôi phục tác vụ

Lợi ích

- Tận dụng CPU tối đa
- Tăng thông lượng hệ thống I/O

Thực tiễn trong Python

- Sử dụng thư viện `asyncio` kết hợp `coroutine` (`async/await`) để cho phép hàng vạn tác vụ chạy xen kẽ đồng thời trên cùng một luồng.

3. Mô hình thực thi bất đồng bộ

3.1. Bản chất cốt lõi

Bản chất cốt lõi

Trọng tâm là **Cơ chế không chặn (Non-blocking)**: Luồng chính (main thread) gửi yêu cầu thực thi tác vụ nền và ngay lập tức tiếp tục xử lý việc khác mà không dừng lại chờ đợi.

Bất đồng bộ có thể sử dụng các hạ tầng thực thi khác nhau tùy thuộc vào loại tác vụ (đơn luồng, đa luồng hoặc đa tiến trình).

3.2. Phân biệt song song truyền thống và bất đồng bộ

Song song truyền thống

Luồng chính chia việc cho luồng/tiến trình con chạy song song, nhưng luồng chính **dừng hoạt động** để chờ đợi kết quả hoàn thành (Bị chặn - Blocking).

Bất đồng bộ có luồng nền

Luồng chính chia việc cho các luồng /tiến trình con chạy dưới nền, và **vẫn tiếp tục chạy độc lập** để xử lý các sự kiện khác (Không bị chặn - Non-blocking).

3.3. Phân loại 3 hình thức thực thi

Mô hình	Bản chất cách chạy	Đặc điểm nổi bật
Đơn luồng (Single-threaded)	Chạy xen kẽ (Interleaved): Các tác vụ luân phiên chạy trên 1 luồng chính qua Event Loop.	Rất nhẹ, tốn ít tài nguyên, loại bỏ rủi ro Race Condition.
Dựa trên Luồng (Thread-based)	Chạy song song nền: Đẩy tác vụ chờ I/O sang Thread Pool dưới nền, giải phóng luồng chính.	Phù hợp tác vụ chờ I/O (API, DB). Luồng nền nhẹ, tối ưu RAM.
Dựa trên Tiến trình (Process-based)	Chạy song song (CPU): Đẩy tác vụ nặng sang Process Pool (CPU core riêng) để xử lý ngầm.	Phù hợp tác vụ CPU-bound. Lách rào cản khóa GIL của Python.

4. Trạng thái CPU trong thực thi

4.1 Xử lý tuần tự

Busy (Bận): CPU xử lý tính toán của tác vụ (render, thuật toán).

Idle (Nhàn rỗi): CPU dừng lại chờ I/O (đọc file, mạng).

→ Gây lãng phí lớn tài nguyên phần cứng.

4.2 Tối ưu hóa CPU trong xử lý đồng thời

Chia nhỏ các tác vụ và sắp xếp đơn xen.

Khi một tác vụ rơi vào trạng thái chờ I/O (Idle), CPU lập tức chuyển sang thực hiện bước tính toán nhỏ của tác vụ khác, giữ CPU luôn ở trạng thái **Busy**.

Xử lý tuần tự

LÃNG PHÍ TÀI NGUYÊN



Tối ưu hóa CPU trong xử lý đồng thời

TÂN DUNG TỐT TÀI NGUYÊN



4.3 Cơ chế Đơn luồng (Single-thread Async)

Treo tác vụ tự nguyện: Khi Coroutine gặp lệnh I/O, nó tự giải phóng quyền kiểm soát và đi vào trạng thái treo dưới sự điều phối của Event Loop.

1. Gửi yêu cầu: Luồng chính gửi yêu cầu xử lý công việc.
2. Treo tự nguyện: Gặp tác vụ chờ đợi (I/O), tác vụ tạm dừng và nhường chỗ.
3. Làm việc khác: Luồng chính chuyển sang chạy tác vụ tiếp theo.
4. Nhận thông báo: Khi I/O hoàn thành, HĐH gửi tín hiệu sẵn sàng.
5. Tiếp tục: Luồng chính khôi phục tác vụ cũ từ vị trí đã dừng và chạy tiếp.

Kết quả: Một luồng duy nhất vẫn có thể giữ CPU luôn bận rộn phục vụ hàng loạt kết nối đồng thời.

4.4 Bất đồng bộ kết hợp Pool

Để tối ưu hóa hiệu năng phần cứng toàn diện, bất đồng bộ phải kết hợp với hai cơ chế nền tảng:

A. Thread-based Async (Cho tác vụ I/O-bound)

Cách chạy: Event Loop đẩy tác vụ chờ đợi I/O (tải file, gọi API) vào các luồng con (Thread Pool).

Trạng thái CPU: Các luồng thay nhau chờ I/O, giải phóng hoàn toàn luồng chính. Trạng thái Idle được giảm thiểu tối đa.

B. Process-based Async (Cho tác vụ CPU-bound)

Cách chạy: Event Loop đẩy tác vụ tính toán nặng sang các tiến trình độc lập (Process Pool). Mỗi tiến trình chạy trên một nhân CPU thực tế.

Trạng thái CPU: Đạt trạng thái Busy đồng thời trên nhiều lõi (True Parallelism), vượt qua rào cản khóa GIL của Python.

QUY TRÌNH HOẠT ĐỘNG VỚI POOL

1

Chuẩn bị (Pool)

Tạo sẵn một nhóm các luồng/tiến trình con ngằm rảnh rỗi chờ việc.

2

Giao việc nền

Luồng chính giao công việc cho nhóm Worker dưới nền xử lý.

3

Nhận Future

Nhận ngay Future theo dõi" và tiếp tục chạy các tác vụ khác.

4

Xử lý độc lập

Một Worker rảnh tự lấy việc ra làm (tiến trình chạy trên lõi CPU riêng).

5

Trả kết quả

Khi xong, worker ghi kết quả vào biên lai và quay lại chờ việc.

6

Nhận dữ liệu

Main kiểm tra biên lai để lấy kết quả tại thời điểm thích hợp.

4.5 Lợi ích hệ thống & Ứng dụng thực tế

Lợi ích hệ thống

Tăng Thông lượng (Throughput): Phục vụ hàng vạn yêu cầu cùng lúc bằng cách tận dụng tối đa tài nguyên.

Giảm Độ trễ (Latency): Thời gian phản hồi nhanh hơn nhờ tác vụ được xử lý ngầm và song song.

Ứng dụng thực tế

Web server hiệu năng cao: FastAPI, Nginx.

Hệ thống thời gian thực: Chat, Streaming.

Xử lý dữ liệu lớn: Kết hợp cả I/O và tính toán.

Tác vụ nền: Crawling, Image/Video Rendering.

Nhận định: "Pool giúp tái sử dụng các worker, giảm chi phí tạo/hủy luồng hoặc tiến trình, từ đó cải thiện Throughput và giảm Latency của hệ thống."

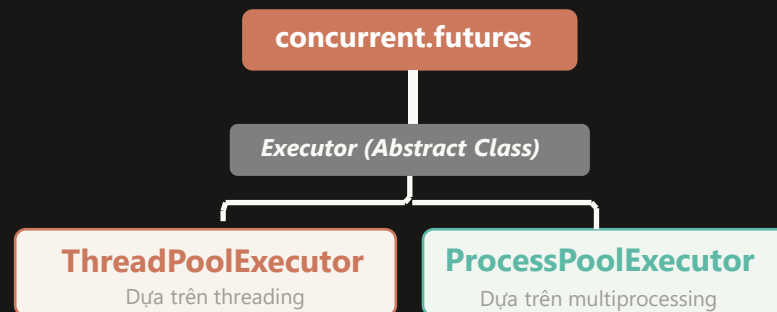
Phần 02

GIỚI THIỆU VỀ MODULE CONCURRENT.FUTURES

ThreadPool, ProcessPool, Futures và Quản lý Task.

1. Tổng quan concurrent.futures

Ra mắt từ phiên bản Python 3.2, cung cấp một Interface cấp cao (high-level interface)



Mục đích cốt lõi

Giúp lập trình viên thực thi các hàm/lệnh gọi một cách bất đồng bộ.

Ưu điểm

Tự động quản lý

Loại bỏ gánh nặng tự quản lý luồng/tiến trình thủ công.

Đơn giản hóa

Giao diện đồng nhất giúp thực thi bất đồng bộ dễ dàng.

Tối ưu đa lõi

Khai thác triệt để sức mạnh CPU cho tác vụ nặng thông qua Pool.

2. Các lớp và các phương thức

2.1. Lớp trừu tượng Executor

Lớp trừu tượng Executor

Bản chất: Là một lớp trừu tượng định nghĩa "giao diện chuẩn" để thực hiện các lệnh gọi bất đồng bộ.

Nguyên tắc sử dụng: **KHÔNG** được khởi tạo trực tiếp (do chưa có logic lỗi). Bắt buộc phải sử dụng thông qua các lớp con cụ thể.

Ba phương thức cốt lõi: **submit**, **map**, và **shutdown**.

2.2. Các phương thức phân phối và quản lý tác vụ

submit()

Giao việc đơn lẻ:

Lên lịch chạy ngầm một tác vụ (non-blocking).

Trả về lập tức đối tượng **Future** ("biên lai" theo dõi tiến độ/kết quả).

map()

Giao việc hàng loạt:

Tự động lặp và thực thi tác vụ đồng thời trên một mảng dữ liệu (VD: áp dụng bộ lọc cho 1.000 hình ảnh).

Trả về bộ lặp (iterator) chứa kết quả. Ném lỗi **TimeoutError** nếu quá hạn.

shutdown()

Giải phóng tài nguyên:

Ra tín hiệu dọn dẹp hệ thống. Nếu cố tình giao thêm việc sẽ báo lỗi **RuntimeError**.

Khuyến cáo: Luôn khởi tạo trong khối `with` để tự động gọi shutdown an toàn.

3. Đối tượng Future (Tương lai)

3.1. Khái niệm đối tượng Future

Khái niệm

Là đối tượng đóng gói việc thực thi bất đồng bộ, đại diện cho kết quả của một tác vụ đang chạy ngầm.

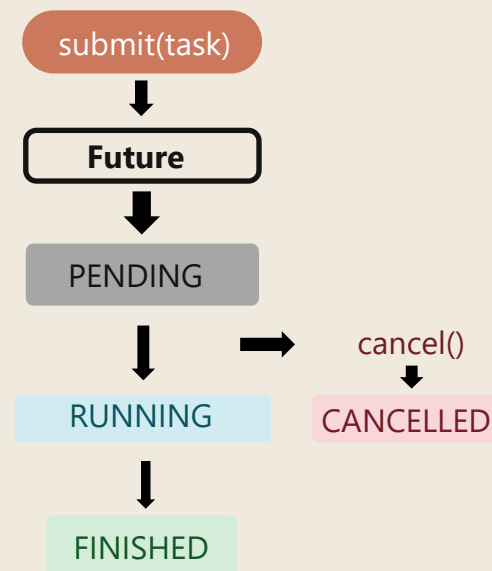


Hình tượng "Tờ biên lai"

Khi giao tác vụ tốn thời gian, hệ thống trả ngay về một "biên lai" (Future) mà không bắt luồng chính phải chờ. Luồng chính dùng "biên lai" này để theo dõi tiến độ, kiểm tra lỗi và lấy kết quả khi tác vụ xong.

Lưu ý khởi tạo: Lập trình viên không tự khởi tạo thủ công (trừ khi testing). Hệ thống sẽ tự động sinh ra và trả về qua phương thức `Executor.submit()`.

Vòng đời Future



```
future = executor.submit(task)
```

3.2 Các phương thức của Future

1. Kiểm tra trạng thái

Trả về **True** nếu tác vụ đang chạy và không thể hủy.

Trả về **True** nếu tác vụ đã hoàn thành hoặc đã bị hủy thành công.

Trả về **True** nếu tác vụ đã được hủy bỏ thành công.

2. Điều khiển & Kết quả

Cố gắng hủy tác vụ trong hàng đợi.

True: Hủy thành công (chưa chạy)

False: Đang chạy hoặc đã xong

Chờ và lấy kết quả. Ném lỗi **TimeoutError** nếu quá hạn.

Tương tự `result()`, nhưng dùng để lấy ra ngoại lệ nếu thất bại.

3. Cơ chế Callback

Khái niệm:

Đính kèm một hàm fn vào đối tượng Future.

Cơ chế hoạt động:

Hàm fn tự động kích hoạt ngay khi tác vụ hoàn thành hoặc bị hủy.

Lợi ích:

Giúp luồng chính **không cần chờ** (non-blocking), chuyển sang kiến trúc phản ứng tự động.

4. Pooling

4.1. Khái niệm

Định Nghĩa: là trình quản lý phần mềm duy trì sẵn một số lượng worker (luồng/tiến trình) rảnh rỗi để phân bổ thực thi tác vụ đồng thời.

Mục đích cốt lõi:

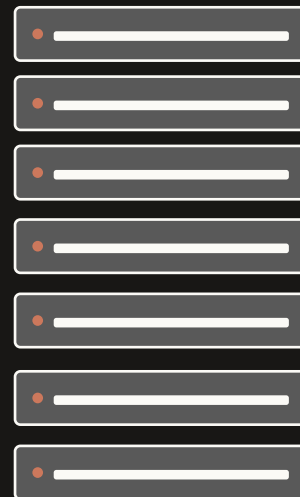
- Tăng hiệu suất hệ thống.
- Tránh độ trễ (latency) do việc tạo mới và hủy luồng/tiến trình liên tục mang lại.

Cơ chế tái sử dụng

- Worker làm xong việc không bị hủy mà được giữ lại để thực hiện tác vụ khác. Số lượng worker tự động điều chỉnh theo phần cứng.

Cơ chế Reuse Worker

Hàng đợi tác vụ



Nhóm worker



"Worker không bị hủy sau khi xong việc, mà quay lại Pool để chờ tác vụ tiếp theo."

4.2. So sánh Executor

Tiêu chí	ThreadPoolExecutor	ProcessPoolExecutor
Worker bên dưới	Luồng (Threads)	Tiến trình (Processes)
Không gian bộ nhớ	Dùng chung bộ nhớ chương trình	Độc lập, không dùng chung bộ nhớ
Rào cản GIL	Bị giới hạn bởi GIL của python	Lách qua được GIL (Đa nhân thực sự)
Loại tác vụ phù hợp	Nặng về I/O (Tải file, gọi API)	Nặng về CPU (Tính toán toán học)
Số lượng worker	$\min(32, \text{CPU} + 4)$	Bằng số lõi CPU logic

4.3. Lưu ý kỹ thuật

ThreadPoolExecutor

- **Deadlocks:** Xảy ra nếu một hàm đang chạy lại đứng chờ kết quả của một Future khác trong cùng một Pool.

ProcessPoolExecutor

- **Yêu cầu Picklable:** Các tham số đối tượng phải có khả năng tuần tự hóa để truyền giữa các tiến trình độc lập được.
- **BrokenProcessPool:** Ngoại lệ nghiêm trọng khi một tiến trình Worker bị chết đột ngột, làm sập toàn bộ Pool

5. Task Queue & Reuse

1. Hàng đợi tác vụ (Task Queue)

- Đóng vai trò bộ đệm lưu trữ các lệnh gọi chưa được thực thi.
- Trình quản lý giám sát Task Queue và phân bổ công việc tuần tự cho các tiến trình/luồng rảnh rỗi trong Pool.

2. Cơ chế Tái sử dụng (Reuse)

Vấn đề cũ: Việc liên tục tạo mới và hủy tiến trình/luồng cho các tác vụ ngăn sinh ra độ trễ (latency) rất lớn.

Giải pháp Reuse: Một tiến trình/luồng không bị tiêu diệt mà được sử dụng nhiều lần cho các tác vụ khác nhau.

Hiệu quả:

- Triệt tiêu độ trễ (latency) cấp phát phần cứng.
- Tự động điều chỉnh số lượng worker khớp với sức mạnh máy tính.

6. Ví dụ: I/O-bound Task

```
from concurrent.futures import ThreadPoolExecutor,
ProcessPoolExecutor
import time
def tai_web(url):
    time.sleep(1)
    return f"Đã tải {url}"
def tinh_toan_nang(n):
    return sum(i * i for i in range(n))
if __name__ == "__main__":
    urls = ["google.com", "facebook.com", "github.com"]
    print("[1. I/O-bound Task]")
    # Chạy Tuần tự (Không dùng Pool)
    start = time.time()
    for url in urls:
        tai_web(url)
    print(f"-> [Tuần tự] tốn: {time.time() - start:.2f}s")
    # Chạy bằng ThreadPool
    start = time.time()
    with ThreadPoolExecutor() as executor:
        list(executor.map(tai_web, urls))
    print(f"-> [ThreadPool] tốn: {time.time() - start:.2f}s")
```

```
# (Tiếp tục phần main...)
print("-" * 50)
inputs = [8_000_000, 8_000_000, 8_000_000, 8_000_000]
print("[2. CPU-bound Task]")
start = time.time()
for val in inputs:
    tinh_toan_nang(val)
print(f"-> [Tuần tự] tốn: {time.time() - start:.2f}s")
# Chạy bằng ThreadPool (Bị khóa bởi GIL)
start = time.time()
with ThreadPoolExecutor(max_workers=4) as executor:
    list(executor.map(tinh_toan_nang, inputs))
print(f"-> [ThreadPool] tốn: {time.time() - start:.2f}s")
# Chạy bằng ProcessPool (Đa nhân thực tế)
start = time.time()
with ProcessPoolExecutor(max_workers=4) as executor:
    list(executor.map(tinh_toan_nang, inputs))
print(f"-> [ProcessPool] tốn: {time.time() - start:.2f}s")
```

Kết quả:

--- BẮT ĐẦU CHẠY THỬ NGHIỆM ĐO THỜI GIAN ---

[1. I/O-bound Task]

-> [Không dùng Pool - Tuần tự] tốn: 3.00 giây

--- BẮT ĐẦU CHẠY THỬ NGHIỆM ĐO THỜI GIAN ---

[1. I/O-bound Task]

-> [Không dùng Pool - Tuần tự] tốn: 3.00 giây

[1. I/O-bound Task]

-> [Không dùng Pool - Tuần tự] tốn: 3.00 giây

-> [ThreadPoolExecutor] tốn: 1.00 giây

-> [Không dùng Pool - Tuần tự] tốn: 3.00 giây

-> [ThreadPoolExecutor] tốn: 1.00 giây

-> [ThreadPoolExecutor] tốn: 1.00 giây

[2. CPU-bound Task]

-> [Không dùng Pool - Tuần tự] tốn: 2.22 giây

-> [ThreadPoolExecutor] tốn: 2.62 giây

-> [ProcessPoolExecutor] tốn: 0.70 giây

Phần 03

Module Asyncio

Nền tảng về tính toán bất đồng bộ và lập trình không chặn.

1. Thành phần chính của asyncio

asyncio là thư viện chuẩn Python (3.4+) cho lập trình các tác vụ đồng thời (concurrent) trên **đơn luồng**.

Cú pháp chính

async / await

Phù hợp với (I/O-bound)

Giao thức mạng (Web Requests, API)

Truy vấn Database (IO latency)

Hệ thống tệp (Đọc/ghi file lớn)

4 Thành phần cốt lõi

Event Loop

1 loop/tiến trình - điều phối mọi tác vụ.

Coroutines

Hàm bất đồng bộ - có thể tạm dừng & quay lại.

Tasks

Bao bọc coroutine - lên lịch thực thi song song.

Futures

Đại diện cho kết quả chưa hoàn thành.

Ưu điểm so với Threading

- 01** Không bị chặn bởi khóa GIL
Chạy trên đơn luồng, tránh cơ chế khóa toàn cục của Python.
- 02** Nhẹ hơn nhiều (Low overhead)
Tiết kiệm RAM và chuyển ngữ cảnh cực nhanh so với OS thread.
- 03** Mở rộng cực tốt (Scalability)
Duy trì hàng vạn kết nối đồng thời mà không treo hệ thống.

2. Vòng lặp sự kiện (Event Loop)

2.1. Khái niệm lập trình sự kiện

- **Lập trình sự kiện** là mô hình lập trình mà luồng thực thi được điều khiển bởi các sự kiện (event) ví dụ: nhấp chuột, gõ phím, nhận dữ liệu từ mạng.
- Trong tính toán bất đồng bộ, chương trình không cần chờ tác vụ hoàn thành mà vẫn có thể tiếp tục thực hiện các công việc khác. Khi một sự kiện xảy ra, bộ xử lý sự kiện tương ứng sẽ được kích hoạt để xử lý sự kiện đó.
- **Sự kiện (Event)** là hành động xảy ra khi người dùng tương tác với chương trình, như nhấn phím hoặc click chuột. Quá trình tạo và xử lý sự kiện được thực hiện bởi ba thành phần chính: **Nguồn sự kiện (Event Source)**, **Trình xử lý sự kiện (Event Handler)** và **Vòng lặp sự kiện (Event Loop)**.

2.2. Các thực thể cốt lõi

Nguồn sự kiện (Event source)

Là nơi khởi tạo và phát đi các sự kiện vào trong hệ thống.

Hàng đợi sự kiện (Event Queue)

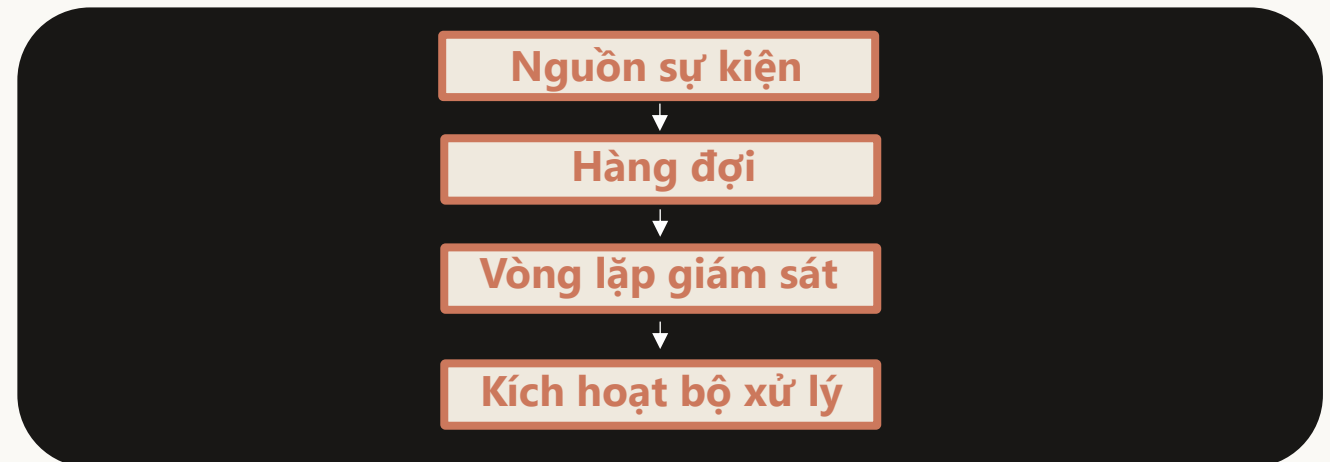
Cấu trúc dữ liệu có nhiệm vụ lưu trữ và xếp hàng các sự kiện vừa phát sinh để chờ đến lượt.

Vòng lặp sự kiện (Event loop)

Cấu trúc điều phối trung tâm hoạt động theo chu kỳ. Có nhiệm vụ liên tục theo dõi hàng đợi và rút từng sự kiện ra để phân phát.

Bộ xử lý sự kiện (Event handler)

Bản chất là một hàm gọi lại (**callback**). Hệ thống sẽ tự động gọi hàm này lên để giải quyết sự kiện, với điều kiện tiên quyết là luồng chính đang rảnh rỗi.



2.3. Chức năng của Vòng lặp sự kiện (Event loop)

Vai trò cốt lõi

Là "trái tim" điều phối bất đồng bộ; bản chất là cấu trúc thiết kế dùng để chờ và gửi sự kiện.

4 cơ chế hoạt động chính

1 Theo chu kỳ

Chạy liên tục xuyên suốt quá trình thực thi để duy trì sự sống cho chương trình.

2 Theo dõi & Thu thập

Liên tục lắng nghe các tương tác (click, gõ phím,...) từ các nguồn cung cấp sự kiện.

3 Xếp hàng (Queueing)

Lưu trữ các sự kiện vừa thu thập được vào một cấu trúc dữ liệu (hàng đợi) để chờ đến lượt.

4 Kích hoạt xử lý

Rút sự kiện từ hàng đợi và gọi hàm phản hồi (callback) tương ứng để giải quyết. Điều kiện: Chỉ thực hiện khi luồng chính (main thread) đang rảnh rỗi.

Cách Event Loop hoạt động

Chờ sự kiện → Lấy ra từ hàng đợi → Gọi trình xử lý tương ứng → Lặp lại chu kỳ.

- Hoạt động liên tục suốt vòng đời chương trình.

Tuyệt đối

Chỉ có 1 event loop hoạt động trên 1 tiến trình.

Lấy và điều khiển Event Loop

```
import asyncio

async def greet():

    await asyncio.sleep(0.5)

    print("👋 Hello from coroutine")

# Lấy (hoặc tạo) một Event Loop

loop = asyncio.get_event_loop() # <- deprecated

# Đặt coroutine vào queue và chạy

loop.create_task(greet()) # đăng ký task

loop.run_until_complete(asyncio.sleep(1))

loop.close()
```

Kết quả Output

Event Loop hiện tại: <ProactorEventLoop
running=True closed=False debug=False>
Xin chào từ Coroutine!

Các phương thức quan trọng

create_task()

call_soon()

call_later()

stop()

is_running()

3. Coroutines - @asyncio.coroutine và async def

3.1. Định nghĩa chi tiết về Coroutine

Coroutine (Đồng chương trình) là một đơn vị xử lý logic lập trình có khả năng **tạm dừng (suspend)** thực thi để nhường quyền kiểm soát cho tác vụ khác, và sau đó **tiếp tục (resume)** lại tại đúng vị trí và trạng thái mà nó đã tạm dừng.

Tính chất cốt lõi:

Cooperative (Hợp tác)

Khác với Thread do Hệ điều hành tự ý ngắt (Preemptive), Coroutine phải **tự nguyện nhường quyền kiểm soát** (trong Python thông qua từ khóa `await`).

Single-threaded Concurrency

Cho phép xử lý đồng thời nhiều tác vụ (concurrency) chỉ trên một luồng duy nhất, giúp **loại bỏ chi phí chuyển đổi ngữ cảnh** đắt đỏ của hệ điều hành và **tránh được các lỗi tranh chấp dữ liệu** (race conditions).

3.2. Bảng so sánh chi tiết

Tiêu chí	Hàm thông thường (Subroutine)	Coroutine (Đồng chương trình)	Thread (Luồng hệ điều hành)
Điểm vào/ra (Entry/Exit)	Chỉ có 1 điểm vào (bắt đầu hàm) và 1 điểm ra (return).	Nhiều điểm vào, điểm ra và điểm tạm dừng (await/yield).	Chạy độc lập từ đầu đến cuối trên một luồng hệ điều hành.
Bảo toàn trạng thái	Trạng thái biến cục bộ bị xóa sạch khi hàm kết thúc.	Trạng thái cục bộ được lưu giữ nguyên vẹn khi tạm dừng.	Trạng thái được lưu trong ngăn xếp (stack) riêng của thread.
Cơ chế điều phối	Chạy tuần tự theo luồng chính của chương trình.	Hợp tác (Cooperative): Code tự nhường quyền điều khiển.	Tranh đoạt (Preemptive): Hệ điều hành (OS) tự ngắt luồng.
Chi phí tài nguyên	Rất thấp (chỉ là gọi hàm thông thường).	Rất thấp: Chỉ cần một lượng nhỏ RAM để lưu trạng thái.	Cao: Mỗi luồng cần phân bổ vùng nhớ stack riêng (ví dụ 1MB-8MB).
Chuyển đổi ngữ cảnh	Do ngôn ngữ/compiler thực hiện (nhanh).	Do ứng dụng / Event Loop quản lý (cực kỳ nhanh).	Do CPU/Hệ điều hành quản lý (chậm, tốn tài nguyên).
Trường hợp áp dụng	Các tính toán tuần tự thông thường.	Tác vụ nghẽn I/O (I/O-bound) như gọi API, đọc DB, tải file.	Tác vụ nặng về tính toán (CPU-bound) như render video, AI.

3.3. Minh họa bằng Code

```
●●● # PHƯƠNG PHÁP COROUTINE (NON-BLOCKING)

async def fetch_data_async(task_id, delay):
    print(f"[Async] Bắt đầu tải Task {task_id} ({delay}s)...")
    await asyncio.sleep(delay)
    print(f"[Async] -> Đã tải xong Task {task_id}!")
    return f"Data {task_id}"

async def run_asynchronous_demo():
    start = time.time()

    task1 = fetch_data_async(1, 2)
    task2 = fetch_data_async(2, 3)
    task3 = fetch_data_async(3, 1)
    # Chạy đồng thời qua event loop
    results = await asyncio.gather(task1, task2, task3)
    print(f"[Async] Tổng thời gian: {time.time() - start:.2f}s")
```

OUTPUT Terminal

--- BẮT ĐẦU CHẠY BẤT ĐỒNG BỘ ---

[Async] Bắt đầu tải dữ liệu cho Task 1 (chờ 2s)...

[Async] Bắt đầu tải dữ liệu cho Task 2 (chờ 3s)...

[Async] Bắt đầu tải dữ liệu cho Task 3 (chờ 1s)...

[Async] -> Đã tải xong Task 3!

[Async] -> Đã tải xong Task 1!

[Async] -> Đã tải xong Task 2!

Tổng thời gian chạy: 3.01 giây

"Kết quả cho thấy các tác vụ được thực hiện xen kẽ. Tổng thời gian chỉ bằng thời gian của tác vụ lâu nhất (3s)."

4. Quản lý Tác vụ - asyncio.Task

Định nghĩa Task

Task: Là lớp con của asyncio, dùng để **đóng gói** và **quản lý** một coroutine.

So sánh

Coroutine chỉ là **bản định nghĩa**

Task = **Định nghĩa** + **Lên lịch chạy** + **Theo dõi trạng thái**.

Phương thức chính:

cancel() done() result() add_done_callback()

***Hủy Task: Khi gọi cancel() -> kích hoạt lỗi CanceledError -> Bắt buộc dùng try/except để dọn dẹp tài nguyên.**

```
async def background_job():
    try:
        print("-> [Task]: Đang thực hiện tác vụ ngầm...")
        await asyncio.sleep(5)
        return "Dữ liệu hoàn thành!"
    except asyncio.CancelledError:
        print("-> [Task]: Đã bắt được sự kiện HỦY!")
        raise

async def main():
    task = asyncio.create_task(background_job())
    await asyncio.sleep(1)
    task.cancel()
    try: await task
    except asyncio.CancelledError:
        print("Đã xác nhận hủy thành công.")
```

KẾT QUẢ OUTPUT

-> [Task]: Đang chạy ngầm...

Hoàn thành chưa? False | Yêu cầu hủy Task...

-> [Task]: Đã bắt được sự kiện HỦY!

Đã xác nhận Task bị hủy bỏ thành công.

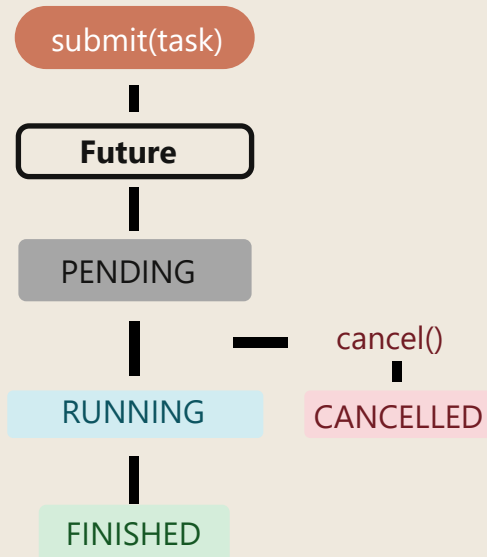
5. asyncio.Future và add_done_callback()

Cơ chế Callback

- Dùng hàm `add_done_callback(fn)` để gắn hành động tự kích hoạt khi xong việc (thành công, lỗi hoặc hủy).
- **Đặc biệt:** Có thể gắn **hiều callback** cùng lúc cho 1 Future hoặc 1 Task.

`result()` `set_result()` `done()` `cancel()` `exception()`

Vòng đời Future



```
def khi_hoan_thanh(future):
    print("-> [Callback]: Nhận tín hiệu hoàn thành!")
    print(f"-> [Callback]: Kết quả = '{future.result()}'")
async def gia_lap_tinh_toan(future):
    print("-> [Coroutine]: Đang tính toán ngầm...")
    await asyncio.sleep(1.5)
    print("-> [Coroutine]: Đã xong! Thiết lập kết quả...")
    future.set_result("KẾT QUẢ THÀNH CÔNG")
async def main():
    loop = asyncio.get_running_loop()
    future = loop.create_future()
    future.add_done_callback(khi_hoan_thanh)
    asyncio.create_task(gia_lap_tinh_toan(future))
    await future
```

KẾT QUẢ OUTPUT

Main đang chờ Future...

```
-> [Coroutine]: Đang xử lý ngầm...
-> [Coroutine]: Đã xong! Thiết lập kết quả...
-> [Callback]: Nhận tín hiệu!
-> [Callback]: Kết quả thu được = 'KẾT QUẢ THÀNH CÔNG'
```

Tổng kết kiến trúc

Non-blocking & Asynchronous

Tối đa hóa hiệu năng CPU bằng cách không đứng chờ các tác vụ I/O.

`concurrent.futures`

Tối ưu Đa luồng (I/O-bound) và Đa tiến trình (CPU-bound) qua ThreadPool và ProcessPool.

`asyncio`

Tối ưu Single-threaded Concurrency thông qua Event Loop và Coroutine, nhẹ và hiệu suất cao.
